Guía del notebook `1_01_mcculloch_pitts`: la neurona de McCulloch y Pitts

Puntos: 3 · **Tiempo:** 20–30 min · **Clase:** 2 · **Archivo:** `notebooks/1_01_mcculloch_pitts.ipynb`
Teoría completa: Guía de la Clase 2, sección B2.

¿Qué vas a aprender?

- Cómo decide la primera neurona artificial de la historia (1943).
- A traducir unas reglas escritas en palabras a una función de Python con `for` e `if`.

Antes de empezar

1. Abre **Anaconda PowerShell Prompt** y escribe `conda activate nni`.
2. Abre VS Code → **File** → **Open Folder** → `curso-redes-neuronales` → carpeta `notebooks` → `1_01_mcculloch_pitts.ipynb`.
3. Arriba a la derecha, elige el kernel **nni**.
4. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**. Las partes que tú completas dicen `TODO`.

> **Sin instalar nada:** en Google Colab, **Archivo** → **Abrir cuaderno** → **GitHub** → `datahuba/curso-redes-neuronales` → este notebook.

La idea en 2 minutos

La neurona se escribe **C(E, I, u)**:

- **E**: lista de entradas **excitatorias** (cada una vale 0 o 1). Empujan a la neurona a encenderse.
- **I**: lista de entradas **inhibitorias** (0 o 1). Si alguna vale 1, la neurona se apaga.
- **u**: el **umbral**: cuántas excitatorias encendidas hacen falta.

Reglas, **en este orden**:

1. Si **alguna** inhibitoria vale 1 → la salida es **0**.
2. Si no, suma las excitatorias: si la suma es **mayor o igual que u** → la salida es **1**.
3. En otro caso → **0**.

Analogía: un portero de discoteca. Las inhibitorias son la "lista negra": si estás ahí, no entras. Si no, entras solo si traes al menos *u* invitaciones (excitatorias).

Paso a paso

Celda "importamos paquetes"

Solo carga NumPy. Ejecútala.

Celda `TODO (2 puntos)`: la función `neuronaMyP(E, I, u)`

Reemplaza `pass` por tu código. **No uses funciones de NumPy**: solo `for`, `if` y sumas.

Pistas:

1. Recorre la lista `I` con un `for`. Si encuentras un 1, haz `return 0` de inmediato.
2. Crea una variable `suma = 0` y súmale cada elemento de `E` con otro `for`.
3. Al final, compara: si `suma >= u` devuelve 1; si no, 0.

Celda "suponga"

Ya trae `E = [1]`, `I = [0]` y `u = 1`. Agrega una línea que imprima `neuronaMyP(E, I, u)`.
✅ **Comprobación:** debe imprimir **1**: no hay inhibición y la suma (1) alcanza el umbral (1).

Celda `TODO (1 punto)`: un programa para cualquier E, I y u

Escribe código que pruebe la neurona con varios casos. Por ejemplo, una función `evaluar(E, I, u)` que imprima la entrada y la salida, y llámala con distintos valores.

✅ **Comprobación** (todos deben coincidir):

Llamada	Qué representa	Salida
`neuronaMyP([], [0], 0)`	NOT con entrada 0	1
`neuronaMyP([], [1], 0)`	NOT con entrada 1	0
`neuronaMyP([1, 1], [], 2)`	AND de (1, 1)	1
`neuronaMyP([1, 0], [], 2)`	AND de (1, 0)	0
`neuronaMyP([0, 1], [], 1)`	OR de (0, 1)	1
`neuronaMyP([1, 1, 0], [], 2)`	"al menos 2 de 3"	1

Errores frecuentes

Síntoma	Causa	Solución
Imprime `None`	Falta un `return` en algún camino	Revisa que la función siempre devuelva 0 o 1
`neuronaMyP([], [0], 0)` da 0	Olvidaste que con E vacío la suma es 0, y 0 ≥ 0	Con `>=` sale 1; revisa que no uses `>`
Una inhibitoria activa no apaga la neurona	Revisas I después de sumar, o no haces `return`	Revisa I **primero** y sal con `return 0`
`NameError: neuronaMyP is not defined`	No ejecutaste la celda de la función	Ejecútala antes de las de prueba